

中山大學
SUN YAT-SEN UNIVERSITY

ROSE——编译原理逆向工程工具

实验三-设计文档

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 6 月

结 束 日 期: 2026 年 6 月

实 验 地 点: B203

目录

1 实验简述 **2**

2 Oberon-0 语法规则回顾 **2**

3 JavaCUP 文法规范设计 **3**

3.1 终结符声明 3

3.2 非终结符声明 3

3.3 优先级与结合性 3

3.4 关键产生式设计 3

4 终结符与 Token 编码对照 **4**

5 JavaCUP 与 Bison 比较 **5**

6 JavaCC 与 JavaCUP 比较 **6**

7 JavaCUP 生成与编译结果 **7**

7.1 批处理脚本说明 7

7.2 生成与编译截图 8

8 语法分析运行结果 **9**

9 实验心得与体会 **10**

1 实验简述

本项目 ROSE 是一个编译原理逆向工程工具，旨在对 Oberon-0 语言源程序进行词法分析、语法分析、语义分析，并自动生成流程图和调用关系图。本实验为第三阶段——使用 JavaCUP 工具自动生成 LALR(1) 语法分析器，实现对 Oberon-0 源程序的语法结构解析，并结合 ROSE 工具集的 CallGraph 库生成过程调用图。

字层（实验二）将字符流转化为 Token 流，语法分析层将 Token 流转化为结构化的语法树，并在此之上执行语义动作——收集过程声明与调用关系，最终以可视化形式呈现调用依赖图。本实验的核心挑战在于：(1) 将 Oberon-0 的 EBNF 语法等价转换为 JavaCUP 的声明式产生式规范；(2) 处理 Token 类型码与 JavaCUP 自动编号间的映射不一致问题；(3) 消除 LALR 分析表中的歧义冲突；(4) 在语义动作中集成 CallGraph 的过程-调用边收集逻辑。

2 Oberon-0 语法规则回顾

Oberon-0 的完整 EBNF 语法定义（摘自 Wirth, 1996）如下。其中 `Module` 为顶层结构，包含声明序列（CONST、TYPE、VAR、过程声明）和语句序列。表达式采用经典的运算符优先级分层。`Designator`（设计子）的选择器是 Oberon 语言族的特色——通过 `.` 字段名和 `[下标]` 实现对 RECORD 和 ARRAY 类型的选择访问。

```
Module     = MODULE ident ";" [ImportList] DeclarationSequence
             [BEGIN StatementSequence] END ident "." .

DeclarationSequence = { CONST {ConstDeclaration ";"} |
                        TYPE {TypeDeclaration ";"}  |
                        VAR  {VarDeclaration ";"} }
                        {ProcedureDeclaration ";" | ForwardDeclaration ";"} .

ConstDeclaration = ident "=" ConstExpression .
TypeDeclaration  = ident "=" Type .
Type             = ident | ARRAY ConstExpression OF Type |
                   RECORD FieldListSequence END .
VarDeclaration   = IdentList ":" Type .
ProcedureDeclaration = PROCEDURE ident [FormalParameters] ";"
                       DeclarationSequence [BEGIN StatementSequence] END ident .
FormalParameters = "(" [FPSection {";" FPSection}] ")" .
FPSection        = [VAR] ident {"," ident} ":" Type .
IdentList        = ident {"," ident} .

StatementSequence = Statement {";" Statement} .
Statement         = [ ident Selector {"," ident Selector} ":=" Expression |
                      ident ["(" [Expression {"," Expression}] ")"] |
                      IF Expression THEN StatementSequence
                           {ELSIF Expression THEN StatementSequence}
                           [ELSE StatementSequence] END |
                      WHILE Expression DO StatementSequence END |
                      Read | Write | WriteLn ] .
```

```
Expression        = SimpleExpression [Relation SimpleExpression] .
SimpleExpression  = ["+"|"-"] Term {AddOperator Term} .
Term              = Factor {MulOperator Factor} .
AddOperator       = "+" | "-" | OR .
MulOperator       = "*" | DIV | MOD | "&" .
Factor            = Number | BoolLit | Designator |
                    "(" Expression ")" | "~" Factor .
Designator        = qualident {"." ident | "[" Expression "]"} .
Relation          = "=" | "#" | "<" | "<=" | ">" | ">=" .
```

3 JavaCUP 文法规范设计

3.1 终结符声明

JavaCUP 要求以 `terminal` 关键字显式声明所有终结符及其类型。本实验声明了 49 个终结符，涵盖 16 个保留字（MODULE, BEGIN, END, IF, THEN, ELSIF, ELSE, WHILE, DO, PROCEDURE, VAR, CONST, TYPE, RECORD, ARRAY, OF）、5 个预定义关键字（INTEGER_TYPE, BOOLEAN_TYPE, Read, Write, WriteLn）、3 种带类型的字面量终结符（Integer INTEGER, Boolean BOOL_LIT, String IDENT）、15 个运算符和 10 个分界符。

JavaCUP 按声明顺序为每个终结符自动分配整数编码（从 2 开始，0 为 EOF，1 为 error）。由于 TokTen.java 中使用的常量编号与 JavaCUP 自动分配的编号不一致，需在 CupScanner 桥接器中通过 HashMap 建立双向映射（49 对）。

3.2 非终结符声明

JavaCUP 要求以 `non terminal` 关键字显式声明所有非终结符及其语义值类型。本实验声明了 35 个非终结符，其中 `module` 为起始符号。表达式类非终结符（expr、simple_expr、term、factor、designator）的语义值类型为 Object（运行时为 String），标识符列表类的类型为 String。

3.3 优先级与结合性

运算符优先级从低到高声明为：`OR < AND < NOT <` 关系 `(<, <=, =, #, >, >=) < +, - < *, DIV, MOD < DOT, LBRACK`。一元加减在 simple_expr 中直接归约，无需虚终结符。`LBRACK` 和 `DOT` 的优先级声明保证了选择器（`[expr]` 和 `.id`）的高紧密度绑定。

3.4 关键产生式设计

模块结构：`module ::= MODULE IDENT ";" decl_seq BEGIN statement_seq END IDENT "."`。顶层语法分析成功后在语义动作中输出过程数与调用图统计信息。

声明序列：`decl_seq` 采用左递归结构，每类声明（CONST、TYPE、VAR、PROCEDURE）独立成子规则。声明列表（const_decl_list、type_decl_list、var_decl_list）采用右递归 + 空产生式，每个声明尾部消耗分号，避免分号处理的歧义。

IF/WHILE 语句：`if_stmt ::= IF expr THEN statement_seq elsif_parts END | IF expr THEN statement_seq elsif_parts ELSE statement_seq END`。全括号结构（IF-THEN-END 配对）消除了 dangling-else 的二义性。`elsif_parts` 为空产生式或递归匹配 ELSIF-THEN 序列。

过程调用与 CallGraph 集成：每次识别到过程调用（有参/无参），语义动作中创建 CallGraph callSite 节点并查找目标过程 key 添加边。过程名 →key 的映射通过 HashMap procNameToKey 维护（本地维护，避免对 CallGraph 受保护字段的直接访问）。

4　终结符与 Token 编码对照

表 1（保留字与预定义关键字）和表 2（字面量、运算符与分界符）列出了 Oberon-0 的全部终结符及其在 Token.java 和 sym.java 中的编码。JavaCUP 的自动编号（sym 常量）与 Token 手动编码不一致，通过 CupScanner 中的 HashMap 建立映射（第 17 至第 65 行，共 49 对）。

表 1: Oberon-0 终结符与 Token / sym 常量编码对照表（一）

分类	名称	词素	Token 码	sym 码
保留字	MODULE	`MODULE`	1	2
	BEGIN	`BEGIN`	2	3
	END	`END`	3	4
	IF	`IF`	4	5
	THEN	`THEN`	5	6
	ELSIF	`ELSIF`	6	7
	ELSE	`ELSE`	7	8
	WHILE	`WHILE`	8	9
	DO	`DO`	9	10
	PROCEDURE	`PROCEDURE`	10	11
	VAR	`VAR`	11	12
	CONST	`CONST`	12	13
	TYPE	`TYPE`	13	14
	RECORD	`RECORD`	14	15
	ARRAY	`ARRAY`	15	16
	OF	`OF`	16	17
预定义关键字	INTEGER_TYPE	`INTEGER`	17	18
	BOOLEAN_TYPE	`BOOLEAN`	18	19
	READ	`Read`	19	20
	WRITE	`Write`	20	21
	WRITELN	`WriteLn`	21	22

表 2: Oberon-0 终结符与 Token / sym 常量编码对照表（二）

分类	名称	词素	Token 码	sym 码
字面量	INTEGER	整数常量	22	23
	BOOL_LIT	TRUE/FALSE	23	24
	IDENT	标识符	24	25
运算符	PLUS	+	30	26
	MINUS	-	31	27
	STAR	*	32	28
	DIV	DIV	33	29
	MOD	MOD	34	30
	AND	&	35	31
	OR	OR	36	32
	NOT	~	37	33
	EQ	=	38	34
	NEQ	#	39	35
	LT	<	40	36
	LE	<=	41	37
	GT	>	42	38
	GE	>=	43	39
分界符	ASSIGN	:=	50	40
	COLON	:	51	41
	SEMI	;	52	42
	DOT	.	53	43
	COMMA	,	54	44
	LPAREN	(	55	45
	RPAREN	)	56	46
	LBRACK	[	57	47
	RBRACK	]	58	48

5 JavaCUP 与 Bison 比较

JavaCUP 与 GNU Bison 同为 LALR(1) 语法分析器生成工具，但面向不同语言生态。表 3 从多个维度进行系统对比。

表 3: JavaCUP 与 GNU Bison 综合对比

维度	JavaCUP	GNU Bison
全称	Construction of Useful Parsers	GNU Parser Generator（Yacc 兼容）
算法	LALR(1)	LALR(1)（默认），可选 IELR(1)、GLR
目标语言	Java	C、C++（可选 Java、D）
输入文件	`*.cup`	`*.y`
输出	Parser.java + sym.java	y.tab.c + y.tab.h
终结符声明	`terminal TOKEN;`	`%token TOKEN`
优先级	`precedence left/right`	`%left / %right`
起始符号	`start with xyz;`	`%start xyz`
语义动作	`{: Java 代码 :}`	`{ C/C++ 代码 }`
语义值传递	`RESULT` 隐式变量	`$$`、`$1`、`$2` 位置引用
词法接口	java_cup.runtime.Scanner	yylex() + yylval 全局变量
错误恢复	无内置错误恢复	`error` 终结符 + yyerror
非终结符声明	必须显式声明（`non terminal`）	可选，自动推导
类型安全	编译期类型检查	运行时检查（union 联合体）
冲突解决	声明式优先级	声明式优先级 + `%expect`

核心差异分析如下：

(1) 语义动作语法：JavaCUP 使用 `{: :}` 括弧 + RESULT 隐式变量传递语义值；Bison 使用 `{}` + `$$`、`$1`、`$2` 位置索引。两者在功能上等价，但 JavaCUP 的标签变量命名（如 `expr:e`）更加可读——可以在语义动作中直接用 `e` 替代位置引用，减少因产生式重构时位置编号变化带来的维护成本。

(2) 非终结符声明：JavaCUP 要求所有非终结符必须显式声明类型和语义值类型；Bison 则允许隐式推断。前者的严格性在大型文法中有助于捕获类型不匹配的错误。

(3) 错误恢复：Bison 提供 `error` 伪终结符支持错误恢复——可在文法中插入” 出错后跳过若干 Token 到同步点” 的规则。JavaCUP 无此机制，语法错误后直接终止分析。这使得工业级编译器中 Bison 更鲁棒，但教学场景下 JavaCUP 的简单行为更易于调试。

(4) 类型安全：JavaCUP 继承 Java 的类型系统，语义值类型在编译期检查。Bison 使用 C 联合体（`%union`），类型错误仅在运行时暴露。

(5) 生态系统：Bison 与 Flex 组成经典的 GNU 编译器工具链，广泛应用于 GCC 等工业编译器。JavaCUP 与 JFlex 组成 Java 体系的对应方案，常见于学术项目和领域特定语言的快速原型开发。

6 JavaCC 与 JavaCUP 比较

JavaCC 采用自上而下的 LL(k) 预测分析，JavaCUP 采用自下而上的 LALR(1) 移进-归约分析。两种范式在文法表达的直观性、左递归处理、冲突解决方式上存在本质差异。表 4 从多个维度进行系统对比。

表 4: JavaCC (LL|top-down) 与 JavaCUP (LALR|bottom-up) 对比

维度	**JavaCC (LL 自上而下)**	**JavaCUP (LALR 自下而上)**
分析方向	自顶向下（Top-down）	自底向上（Bottom-up）
可处理文法类	LL(k) 文法	LALR(1) 文法
左递归	必须消除，需改写为右递归或迭代	原生支持，无需改写
表达式优先级	通过嵌套非终结符层次隐式编码	通过声明式优先级指令显式声明
词法语法耦合	词法和语法在同一.jj 文件	词法（JFlex）和语法（CUP）分离
语义动作	直接在产生式右部插入 Java 代码	用 `{: :}` 插入，支持标签命名
错误消息	详细，提示期望的符号集合	偏技术性，显示冲突状态编号
错误恢复	内置同步恢复，可一次报告多错误	无恢复机制，一次错误即终止
代码组织	单一文件	词法/语法分文件，模块化更好
学习曲线	较陡——需理解 LL、lookahead 谓词	中等——理解 shift/reduce 即可入门
冲突	无冲突概念，直接拒绝有二义性的文法	需解决 shift-reduce / reduce-reduce 冲突

核心差异分析如下：

(1) 左递归是分水岭：LL 分析器遇左递归会无限递归，用户必须改写文法为右递归或 EBNF 迭代。例如 Oberon-0 中 `decl_seq ::= decl_seq decl | empty` 在 JavaCC 中需改为 `decl_seq ::= (decl)*`。JavaCUP 原生支持左递归，文法更贴近原始 BNF，维护更直观。

(2) 表达式优先级表达：JavaCUP 通过 `precedence left/right` 指令显式声明，一行解决。JavaCC 必须通过嵌套非终结符层次（如 Factor → Term → SimpleExpr → Expr）隐式编码——文法结构更冗长，且优先级变更需重构产生式层次，容易引入意外歧义。

(3) 模块化与关注点分离：JavaCC 将词法和语法写在同一.jj 文件中，不利于模块化复用。JavaCUP 将词法（JFlex）和语法（CUP）完全分离，通过 Scanner 接口接驳——体现了编译器设计中" 前端管道"（词法 → 语法 → 语义）的清晰分层。

(4) 冲突解决哲学：JavaCUP 在生成期报告 shift-reduce 和 reduce-reduce 冲突，要求用户通过声明优先级或重构文法来消除歧义——冲突的存在本身就是文法设计的反馈信号。JavaCC 则通过 LOOKAHEAD 谓词让用户在产生式级别指定前瞻深溯，机制不同但都需要对底层分析算法有深入理解。

(5) 错误恢复：JavaCC 在错误恢复方面有天然优势——LL 分析器知道" 当前期望哪些 Token"，可以智能跳过直到找到同步点，从而实现单轮分析中的多错误报告（IDE 友好）。JavaCUP 的 LALR 分析在语法错误后直接终止，最多报告一个错误。

(6) 小结：在 Oberon-0 这类教学语言上，JavaCUP 的自然左递归和声明式优先级使得文法编写更为简洁。但在需要友好错误提示和多错误报告的工业 IDE 场景中，JavaCC 的 LL 恢复机制更为成熟。

7 JavaCUP 生成与编译结果

7.1 批处理脚本说明

实验三提供四支批处理脚本：

- `gen.bat` ——调用 JavaCUP 生成器，从 `oberon.cup` 生成 `Parser.java`（LALR 语法分析器）和 `sym.java`（符号常量类）。符号类命名为 `sym` 而非 `Symbol`，避免与 `java_cup.runtime.Symbol` 命名空间冲突。
- `build.bat` ——编译所有 Java 源文件到 `bin/`。classpath 包含 `java-cup-11b-runtime.jar`、`callgraph.jar` 和 `jgraph.jar`。
- `run.bat` ——运行语法分析器解析指定的 Oberon-0 源文件，语法分析通过后自动显示 CallGraph 调用图窗口。
- `test.bat` ——运行单个测试用例（语法错误变异文件）。
- `gendoc.bat` ——调用 `javadoc` 对 `src/` 下所有手写 Java 源码（CupScanner.java、OberonParserTest.java、Token.java 及 5 个异常类）生成 HTML API 文档，输出至 `doc/` 目录。classpath 包含 `java-cup-11b-runtime.jar`、`callgraph.jar` 和 `jgraph.jar`。

7.2　生成与编译截图

图 1 展示了 `gen.bat` 的执行结果：0 错误、0 警告、87 条产生式、35 个非终结符、49 个终结符、182 个 LALR 状态、0 冲突（超出预期的 0 个）。

图 2 展示了 `build.bat` 的执行结果：Parser.java、CupScanner.java、OberonScanner.java、Token.java 及 5 个异常类全部编译通过。

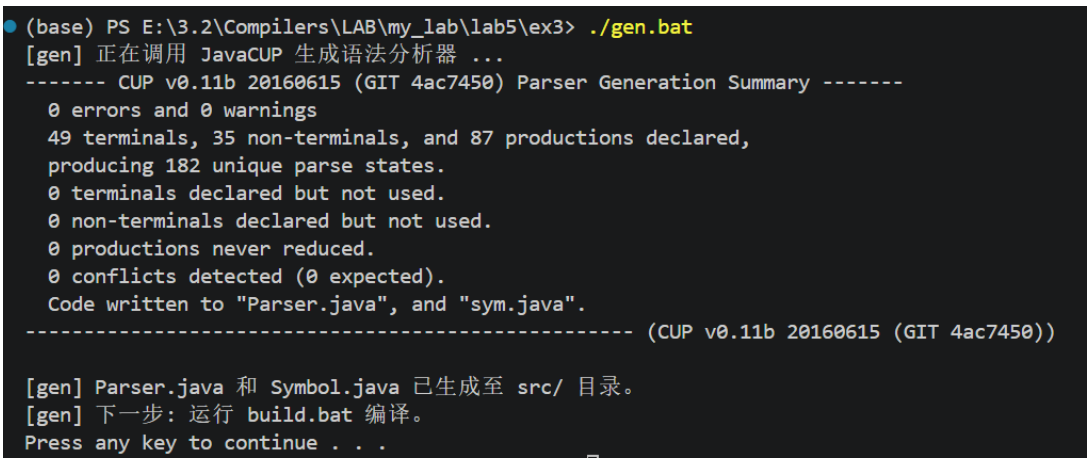


图 1: gen.bat 执行结果：0 错误 0 警告，生成 182 个 LALR 状态

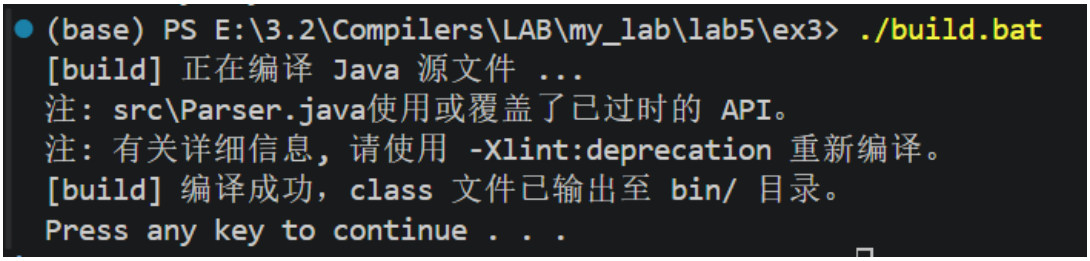


图 2: build.bat 执行结果：所有 Java 源文件编译通过

8　语法分析运行结果

图 3 展示了 `run.bat` 解析 `StudentStats.obr` 的终端输出：语法分析通过，模块 `StudentStats` 分析完成。CallGraph 统计显示 6 个过程（CalcTotalAndAvg、ReadStudent、FindBestStudent、CheckAllPass、PrintStudent、PrintAll）、6 个调用点和 6 条调用边。

图 4 展示了 CallGraph 弹出的可视化调用图窗口，清晰呈现了 6 个过程节点之间的调用依赖关系。

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex3> .\run.bat ..\ex1\StudentStats.obr
[run] 正在解析: ..\ex1\StudentStats.obr
========================================
  ROSE 实验三 -- Oberon-0 语法分析
  源文件: ..\ex1\StudentStats.obr
========================================

[语法分析] 开始解析...
[语法分析] 模块 'StudentStats' 分析完成。
[调用图] 过程数: 6, 调用点数: 6, 边数: 6

=== 语法分析通过，无语法错误！ ===

[调用图] 显示中...
```

图 3: StudentStats.obr 语法分析通过——6 过程、6 调用点、6 边

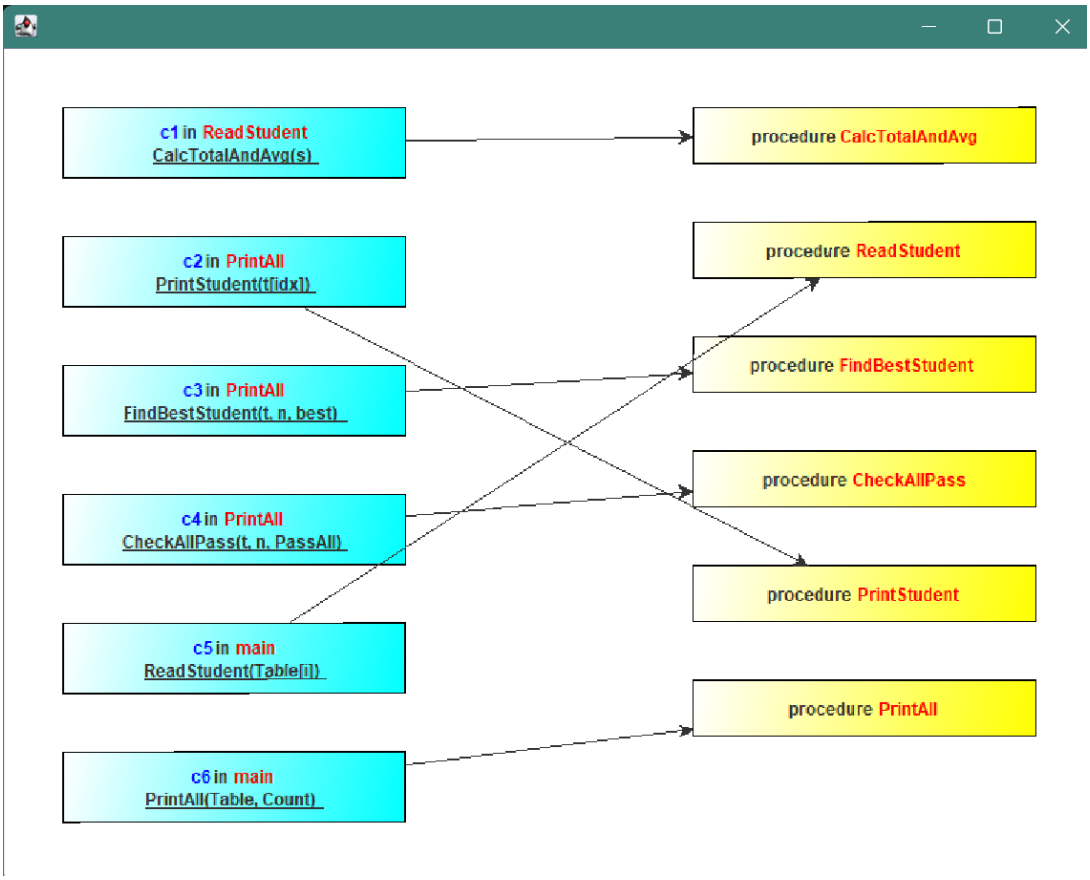


图 4: CallGraph 调用图可视化——6 个过程节点的调用依赖关系

9 实验心得与体会

- **LL 与 LR 范式的直观对比。**实验三（LALR 自动生成）与实验四（递归下降手写）覆盖了两种主流语法分析范式，使我对 shift-reduce 与递归下降的决策机制差异有了具体认知。LALR 无冲突分析表（182 状态，0 冲突）的生成过程，验证了自底向上方法对 Oberon-0 文法的完美适配。

- **非终结符显式声明 = 编译期类型检查。**JavaCUP 要求所有非终结符前置声明，初看冗余，实则提供了" 类型检查" 功能——产生式右部与左部 `RESULT` 类型不匹配时在生成期即报警，避免了到 javac 阶段才发现语义值传递错误。这是声明式工具链" 提早暴露错误" 哲学的体现。

- **Token→sym 编码映射是最隐蔽的集成陷阱。** JavaCUP 按声明顺序重编号终结符，与 Token.java 中手动常量完全不同。若不通过 HashMap 桥接（49 对映射），分析器首行即报语法错误，而 Token 流调试输出完全正确。这揭示了自动生成工具隐含约定的集成风险。

- **语义动作嵌入语法规则 = 语法制导翻译的工程实践。**在过程声明处调用 `graph.addProcedure()`、过程调用处调用 `graph.addEdge()`，语义动作精确嵌入在归约节点上，LALR 的归约时机恰是所有子结构就绪的" 信息汇总点"——这正是语法制导翻译（SDT）的核心思想。

- **工具选择决定架构上限，文法规则决定质量下限。** JavaCUP vs Bison、JavaCC vs JavaCUP 的对比表明：工具提供了分析表压缩、类型安全、Unicode 支持等性能保障（上限），而文法编写质量决定了能否正确解析语言（下限）。